Ridge, Lasso and glmnet

speaking script · MATH 4432 Statistical Machine Learning

Target length **30 minutes**, 18 slides. Body text is what you say; the ruled grey lines are stage directions, not words.

Pronunciation

name	IPA	say it like
glmnet	/ˌdʒiː ɛl ɛm ˈnɛt/	G-L-M-net
lasso	/læˈsuː/ or /ˈlæsoʊ/	la-SOO or LASS-oh
Tibshirani	/ˌtɪbʃɪˈrɑːni/	tib-shi-RAH-nee
Hastie	/ˈheɪsti/	HAY-stee
Ewing	/ˈjuːɪŋ/	YOO-ing
sarcoma	/sɑːrˈkoʊmə/	sar-KOH-muh
rhabdomyosarcoma	/ˌræbdoʊˌmaɪoʊsɑːrˈkoʊmə/	RAB-doh-MY-oh-sar-KOH-muh
neuroblastoma	/ˌnjʊəroʊblæˈstoʊmə/	NURE-oh-blass-TOH-muh
Burkitt	/ˈbɜːrkɪt/	BUR-kit
lymphoma	/lɪmˈfoʊmə/	lim-FOH-muh

Timing

#	slide	length	ends at
1	Title	0:30	0:30
2	Too many predictors	2:15	2:45
3	Two ways to fix it	1:15	4:00
4	Ridge regression	2:15	6:15
5	Shrinking trades bias for variance	1:45	8:00
6	The lasso	1:45	9:45
7	Why the lasso gives exact zeros	2:00	11:45
8	Choosing λ by cross-validation	2:00	13:45
9	Ridge or lasso?	1:45	15:30
10	The elastic net	1:15	16:45
11	glmnet: one package, the whole family	1:45	18:30
12	glmnet in R, the same in Python	2:00	20:30
13	1 · the lasso with cross-validation	1:45	22:15
14	2 · least squares, ridge, lasso, elastic net	2:00	24:15
15	Application: which tumour is it?	2:00	26:15
16	3 · the multinomial lasso	2:00	28:15
17	Take away	1:15	29:30
18	References	0:30	30:00

If you are running late

- **Slide 5, bias and variance** — cut to 1 min – the two curves and the minimum
- **Slide 7, exact zeros** — cut to 1 min – circle versus diamond, corners on the axes
- **Slide 12, R and Python** — cut to 1 min – the swapped names only
- **Slide 14, comparison** — cut to 1 min – point at the output table, one sentence
- **Slide 16, tumour code** — cut to 45 s – point at the output line only

- **Slides 2 to 11, the big picture** — protect them – they are the point of this tutorial

SLIDE 1

Title

0:30
ends at 0:30

Good afternoon. Today we review **ridge regression** and the **lasso**, and then we use them on real problems with **glmnet**, the standard package for these methods.

The first half is the big picture: why we need them, what they do, and how to choose the one number that controls them. The second half is code.

> **STAGE** *First time you say glmnet: "G-L-M-net". It stands for generalized linear models with the elastic net penalty.*

SLIDE 2

Too many predictors

2:15
ends at 2:45

Let us start with the problem. This is a simulation, so we know the truth. We have a small data set, and the response depends on only a **few** predictors. Then we keep adding predictors that are **pure noise** – they have nothing to do with the response – and fit least squares each time.

The dark line is the **training** error. It goes down every time we add a predictor, all the way to almost zero. Least squares always fits the training data better when you give it more columns, even useless ones.

The red line is the **test** error, on new data. It goes up, slowly at first, and then it explodes as the number of predictors gets close to the number of observations. The model has learned the noise.

And when the number of predictors **p** is larger than the number of observations **n**, least squares does not even have a unique answer: infinitely many coefficient vectors fit the training data perfectly.

This is not a strange situation. In genetics you may have thousands of genes and only a hundred patients. We will see such a data set at the end.

> **ASK THE ROOM** *Why does the training error always go down when we add a predictor? (Least squares can always set the new coefficient to zero, so it can never do worse.)*

SLIDE 3

Two ways to fix it

1:15
ends at 4:00

There are two families of fixes.

Subset selection, from the lecture: keep some predictors and drop the rest. Each coefficient is either in the model or out of it. The difficulty is the search: with many predictors there are far too many subsets to try.

Shrinkage, today: keep every predictor, but pull all the coefficients towards zero. There is one dial, called **lambda**, that says how hard we pull.

Both ideas do the same thing in the language of T02: they accept a little **bias** in exchange for a big reduction in **variance**.

SLIDE 4

Ridge regression

2:15
ends at 6:15

Here is ridge regression. The first part is the usual residual sum of squares, the least-squares criterion. The second part, in blue, is the **penalty**: lambda times the sum of the squared coefficients.

So ridge looks for coefficients that fit the data well **and** are small. Lambda decides the balance. When lambda is zero, the penalty disappears and we get least squares. When lambda is very large, the only way to keep the criterion small is to make every coefficient close to zero.

The picture is the **coefficient path** on the Hitters data from the textbook: the salaries of baseball players, with predictors such as hits, walks, and career totals. Each line is one coefficient as lambda grows from left to right. On the left, least squares gives some huge coefficients, some positive and some negative, because the career statistics are strongly correlated with each other. As lambda grows, all of them shrink smoothly towards zero.

Two details. First, we **standardize** the predictors before fitting, so that each has standard deviation one; otherwise the penalty would depend on the units, for example whether a distance is measured in metres or in kilometres. Second, the intercept is **not** penalized; we only shrink the slopes.

SLIDE 5

Shrinking trades bias for variance

1:45
ends at 8:00

Why does shrinking help? Here is another simulation, with almost as many predictors as observations. We repeat the experiment many times, so we can measure the bias and the variance directly, like in T02.

On the left, lambda is almost zero: that is least squares. The squared bias, in blue, is almost zero, but the variance, in green, is huge, and so is the test error.

As lambda grows, the variance falls very fast and the bias grows only slowly. At the best lambda, the red dot, the test error is about seven times smaller than for least squares. That is the whole trade: a little bias for a lot less variance.

If lambda is too large, we shrink too much, the bias takes over and the error goes up again.

SLIDE 6

The lasso

1:45
ends at 9:45

The lasso changes one thing: instead of the **squares** of the coefficients, it penalizes their **absolute values**.

That small change has a big effect. Look at the path, again on Hitters. The coefficients do not just shrink: they hit **exactly zero**, one after another. As lambda grows, fewer and fewer predictors are left, and for a large lambda, none at all.

So the lasso **selects variables**. It does shrinkage and subset selection at the same time. The model it gives is **sparse** – only a few predictors – and much easier to explain. On Hitters, the predictors that survive longest are career runs, career RBI, hits and walks.

> **NOTE** *The lasso is also called L1-penalized regression; ridge is L2-penalized.*

SLIDE 7

2:00
ends at 11:45

Why the lasso gives exact zeros

Why exact zeros for the lasso but not for ridge? We can write both methods in an equivalent form: minimize the residual sum of squares, **subject to a budget** s on the size of the coefficients.

With two coefficients we can draw it. The dark dot is the least-squares estimate, and the ellipses around it are contours: every point on one ellipse has the same residual sum of squares. The shaded region is the budget. We grow the ellipses until they first touch the region; that point, in orange, is the solution.

The ridge region is a **circle**. It is round, so the ellipse usually touches it at a point where both coefficients are non-zero.

The lasso region is a **diamond**, with corners sitting **on the axes**. The ellipse very often touches a corner first, and at a corner one of the coefficients is exactly zero. In many dimensions the diamond has many corners and edges, so many coefficients become zero.

SLIDE 8

2:00
ends at 13:45

Choosing λ by cross-validation

How do we choose lambda? With **cross-validation**, exactly as in T07. For each value of lambda, we fit the lasso with one fold left out, measure the error on that fold, and average over the folds. The red dots are the cross-validation error; the grey bars are one standard error. The numbers at the top count the predictors in the model.

There are two standard choices. **Lambda min** has the smallest cross-validation error.

But look how flat the curve is. Many larger lambdas are just as good, given the uncertainty. So glmnet also reports **lambda one-SE**: the **largest** lambda whose error is within **one standard error** of the minimum. It gives the simplest model that we cannot distinguish from the best one. Here it keeps about half as many predictors as lambda min.

Lambda min is a good default for prediction. Lambda one-SE is popular when you want a small model that you can explain.

> **ASK THE ROOM** *Why the largest lambda, not the smallest, within one standard error? (Larger lambda means a simpler model.)*

SLIDE 9

1:45
ends at 15:30

Ridge or lasso?

So which one should we use? Neither always wins. Another simulation, with the same total

signal in two situations. On the left, only **two** predictors matter. On the right, **all** of them matter a little.

When only two matter, the lasso wins, because it can drop the useless predictors. When all of them matter, ridge wins, because the lasso throws away predictors that carry real signal.

Both are far better than least squares: its test error, in the caption, is many times larger.

In real data we do not know which situation we are in. So we let cross-validation compare them – or we mix them, which is the next slide.

SLIDE 10

1:15

ends at 16:45

The elastic net

The **elastic net** uses both penalties at once. A second number, **alpha**, between zero and one, sets the mix. Alpha equal to one is the lasso; alpha equal to zero is ridge; values in between give a mixture.

The picture shows the three budget regions. The elastic net still has corners on the axes, so it still sets coefficients to exactly zero. But its sides are curved like ridge, and this helps with **correlated** predictors: the lasso tends to keep one of a group of correlated predictors and drop the others, while the elastic net tends to keep the group together.

SLIDE 11

1:45

ends at 18:30

glmnet: one package, the whole family

This brings us to **glmnet**. It solves this one problem: an average loss over the data, plus lambda times the elastic net penalty.

The **family** argument chooses the loss. For a numeric response, gaussian, the loss is the squared error, and we get ridge, lasso or elastic net regression. For two classes, binomial, the loss is minus the log-likelihood of logistic regression. Multinomial does the same for several classes, and there are also Poisson for counts and Cox for survival times. So one recipe covers regression and classification.

Two practical points. glmnet fits the **whole path** – a long grid of lambda values, from the empty model to almost least squares – in one call, and very fast. And it **standardizes** the predictors for you, then reports the coefficients on the original scale.

glmnet is an R package, written by Friedman, Hastie /ˈheɪsti/ and Tibshirani /ˌtɪbʃɪˈrɑːni/; Hastie /ˈheɪsti/ and Tibshirani /ˌtɪbʃɪˈrɑːni/ are also two of the authors of our textbook. Today we use Python, so let us see how the two match.

SLIDE 12

2:00

ends at 20:30

glmnet in R, the same in Python

On the left is glmnet in R. `model.matrix` builds the predictor matrix and turns the factors into zero-one dummy variables. `glmnet` fits the path; `cv.glmnet` does cross-validation; `coef` with `lambda.1se` gives the coefficients at lambda one-SE.

On the right is the same in Python with scikit-learn. `lasso_path` fits the path, `LassoCV` does the

cross-validation, `ElasticNet` is the elastic net, and `LogisticRegression` with an L1 penalty is the multinomial lasso.

Three warnings, at the bottom. First, **the names are swapped**. glmnet's alpha, the mixing number, is called `l1_ratio` in scikit-learn, and glmnet's lambda is called `alpha`. For logistic regression, scikit-learn uses C, which is one over n times lambda: a **small** C means a **strong** penalty.

Second, scikit-learn does **not** standardize for you, so we put a `StandardScaler` in front.

Third, for the lasso, the two packages solve exactly the same problem: I checked that the same lambda gives the same coefficients. For ridge, glmnet scales lambda differently, so the lambda values do not carry over. In both packages, let cross-validation choose.

> **NOTE** *For the gaussian family glmnet divides y by its standard deviation before fitting, which changes the meaning of lambda for the ridge part of the penalty. The R versions of today's analyses are in* `T10_glmnet.R`.

SLIDE 13

1:45
ends at 22:15

1 · the lasso with cross-validation

Now the code, on the Hitters data. `get_dummies` turns the factors into dummy variables. The pipeline first standardizes, then runs `LassoCV` with ten folds.

`LassoCV` gives lambda min directly, as `alpha_`. For lambda one-SE we need three lines: the mean cross-validation error for each lambda, its standard error, and then the largest lambda whose error is within one standard error of the minimum.

In the output, lambda min keeps thirteen predictors and lambda one-SE keeps only six: hits, walks, career runs, career RBI, putouts and the division. These match glmnet's `lambda.min` and `lambda.1se`; R picks the same six predictors.

> **NOTE** *The exact lambda values depend on the random folds; R's cv.glmnet uses its own folds and gives slightly different lambdas, but the same six predictors at lambda.1se.*

SLIDE 14

2:00
ends at 24:15

2 · least squares, ridge, lasso, elastic net

Which method predicts best on Hitters? A fair comparison: we split the players in half at random, choose lambda by cross-validation on the training half only, and measure the error on the test half. We repeat this for twenty different splits and average, because one split is too noisy.

The output shows the test error and p, the number of predictors used. Every model is much better than just predicting the average salary. Least squares is the worst of the four. Ridge and the elastic net are the best. The lasso is in between, but it drops several predictors.

So on this data every penalty beats least squares, and the lasso buys a simpler model for a small loss in accuracy.

SLIDE 15

Application: which tumour is it?

2:00
ends at 26:15

Now an application where glmnet really shines: more predictors than observations.

These data, from Khan and colleagues, are gene expression measurements: more than two thousand genes for each tumour. The tumours are four kinds of childhood cancer that look alike under the microscope: Ewing /ˈjuːɪŋ/ sarcoma /sɑːrˈkoʊmə/, rhabdomyosarcoma /ˌræbdoʊˌmaɪoʊsɑːrˈkoʊmə/, neuroblastoma /ˌnjʊəroʊblæˈstoʊmə/ and Burkitt /ˈbɜːrkɪt/ lymphoma /lɪmˈfoʊmə/. But we have only a few dozen tumours for training, and a small test set.

So p is much larger than n. Least squares, or plain logistic regression, has no unique answer here.

The **multinomial lasso** fits one coefficient vector for each type of tumour, with the lasso penalty. It keeps only about **twenty** of the two thousand genes, and it classifies **every** test tumour correctly.

The picture shows those genes for the training tumours, grouped by type. Red is high expression, blue is low. You can see blocks: each type has a few genes that are switched on or off. That is what makes the lasso useful in biology: it does not just predict, it points to a short list of genes worth studying.

SLIDE 16

3 · the multinomial lasso

2:00
ends at 28:15

The code. We try a grid of lambda values and turn each one into scikit-learn's C, one over n times lambda. `LogisticRegression` with `l1_ratio` equal to one is the lasso penalty, and the `saga` solver supports it for several classes.

`GridSearchCV` chooses lambda by cross-validation accuracy. Lambda goes from large to small, so if several values tie, it keeps the first, the largest lambda, which uses the fewest genes.

This solver converges slowly here, so the search uses a loose tolerance, and then we refit the chosen model with a tight one. If you skip the refit, many coefficients are tiny but not exactly zero, and you count too many genes.

The output: the lambda, the number of genes, and a test accuracy of one – every test tumour correct. R's `cv.glmnet` picks a slightly smaller lambda with its own folds, keeps a couple more genes, is also perfect, and needs less than a second. And at the same lambda, scikit-learn and glmnet pick exactly the same genes.

> **NOTE** *Several lambdas tie at perfect CV accuracy; the tie goes to the largest. Needs scikit-learn 1.8 or newer: older versions need penalty="l1" in LogisticRegression.*

SLIDE 17

Take away

1:15
ends at 29:30

To sum up. With many predictors, least squares overfits; shrinking the coefficients trades a

little bias for much less variance. Ridge, with squares, shrinks every coefficient; the lasso, with absolute values, sets some of them exactly to zero and so selects variables. Standardize the predictors first, and choose lambda by cross-validation: lambda min for prediction, lambda one-SE for a simpler model. Neither method always wins, and the elastic net mixes them. And glmnet gives one recipe for numbers, classes, counts and survival times, even when there are more predictors than observations.

SLIDE 18

0:30

ends at 30:00

References

The reading is the ISL chapter on linear model selection and regularization, and its lab uses the same Hitters data. The glmnet paper is by Friedman, Hastie /ˈheɪsti/ and Tibshirani /ˌtɪbʃɪˈrɑːni/. Every number today comes from the notebook, and `T10_glmnet.R` does the same in R. Questions?

STAGE *Leave this slide up.*

Questions students actually ask

Why must we standardize?

The penalty treats every coefficient the same. If income is measured in dollars instead of thousands of dollars, its coefficient becomes a thousand times smaller and is hardly penalized. Standardizing puts all predictors on the same scale, so the penalty is fair. Least squares does not need it, because its fit does not depend on the units.

Why is the intercept not penalized?

The intercept only sets the overall level of the response. Shrinking it towards zero would make the predictions depend on where zero is on the response scale, which makes no sense.

Should I use lambda min or lambda one-SE?

Lambda min for the best prediction; lambda one-SE when you prefer a smaller model whose cross-validation error is within the noise of the best. The smaller model usually gives up a little accuracy, so it is a real trade-off.

Can I get p-values or confidence intervals for the lasso coefficients?

Not from the usual formulas: the variables were chosen using the same data, so standard errors from least squares are too optimistic. The bootstrap (T08) can show how unstable the selection is; proper inference after selection is an active research area.

Is there a Bayesian view?

Yes. Ridge is the posterior mode when the coefficients have a normal prior centred at zero; the lasso is the posterior mode with a Laplace (double-exponential) prior, which has a sharp peak at zero. ISL section 6.2.2.

Why are R's and Python's lambdas different for ridge?

For the gaussian family, glmnet divides y by its standard deviation before fitting. For the lasso this does not change which coefficients you get at a given lambda; for ridge it does. So compare models through cross-validation, not through raw lambda values.

What happens with ridge logistic regression on the tumour data?

With a suitable lambda it also classifies the test tumours well, but every gene gets a non-zero coefficient, so it does not tell you which genes matter. Try `l1_ratio=0` in the notebook.